

Project Diary — Jairaj Berry

Enrollment A2345923013, Roll 13. Group 298, B.Tech 7CSE (Evening), Amity School of Engineering and Technology. Project MAESTRO — Design and Evaluation of a Safe Multi-Agent System for Natural Language-Driven Desktop Task Automation. Guide: Dr. Rajni Sehgal Kaushik. My role in the team is execution and platform. I own the part of the system that actually touches the computer, the layer that has to work the same way on Windows and on macOS, and the testing, which in a project about safety is not a formality.

Week 1

My first week was spent on the practical question of whether we can build this at all on the machines we own. I checked what we have available on both operating systems, installed the toolchain, and got a language model running locally so that we would know early whether it is fast enough to be usable rather than finding out in week nine. I then went through every tool we plan to use and wrote down its licence and its cost, which is how we can put zero against the bill of material and defend it if asked, because a claim of no cost is only worth something if somebody has actually checked each item. I also set up the folder layout with Shashank, and we agreed at that point on something that has shaped my work since, which is that only one small part of the system is allowed to know which operating system it is running on.

Week 2

While the others were reading about models I read about the input side, because the instruction has to be understood before anything can be planned. I went through intent classification and entity extraction, which is the work of deciding what the user is asking for and pulling out the specific pieces such as the file type, the source folder and the destination. I also tried speech input using a locally running transcription model, since we want the system to be usable by voice without sending audio anywhere, and it worked well enough to keep in scope. The rest of the week went into measuring how long the local model takes to respond on our own hardware. It is not fast, but it is usable, and knowing the real number stopped us designing something that would only run on a machine we do not have.

Week 3

This was my favourite week so far because it was hands-on. Rather than only reading about the existing frameworks, I installed OpenClaw, Hermes and Open Interpreter and used them, and I deliberately gave them the sort of vague instruction a normal person would give, such as asking them to clean up a folder. What I saw went straight into Seenu's comparison. None of them showed me what they were about to do in any useful detail. None of them offered a way to reverse what they had done. In one case the system decided on its own what cleaning up meant and I was glad I was working inside a folder full of test files. Watching it happen is different from reading that it happens, and it convinced me that the preview step we were designing is

the centre of the project and not a nice extra.

Week 4

The gap analysis needed evidence and not just opinion, so I spent the first part of this week reproducing the unsafe behaviour on purpose and writing down exactly what each system did, so that our claims can be backed by something we have seen rather than something we have read. After that I moved on to my own part of the design. We agreed on a closed list of the operations the system is allowed to perform, which means an instruction that would require anything outside that list is rejected rather than improvised, and I began writing the first file operations against it. The rule I followed for each one is that before it can be used it must be able to describe what it would do without doing it, and it must be able to undo itself afterwards. Deleting is the clearest example, because in our system a deletion always moves the file to the trash and never removes it outright.

Week 5

This week I finished the first set of file operations and then spent most of my time trying to break them, which is the part of the work I am responsible for. The test suite now has forty-eight tests passing and the three I care about most are the ones that attack the system rather than exercise it. The first tries to escape the permitted folder using a path that climbs back out of it, and it also tries the same thing through a shortcut that points somewhere it should not, and both are refused before any matching happens. The second gives the system a plan that claims to be harmless while asking for something dangerous, and the fixed scoring rules ignore the claim. The third edits the log after the fact, and the chain of hashes breaks so the change is visible. I also added a check that fails the build if any part of the program outside the execution layer tries to ask which operating system it is on, because that is how a cross-platform project quietly turns into two separate projects.

Where I stand

The execution layer works on macOS for the file operations we need, everything it does can be previewed and reversed, and the tests are in place and passing. The next stretch of work is mine to deliver as well: the browser and voice operations, then the test harness that will run the full benchmark automatically, and after that the Windows version of the execution layer, which is the largest single piece of the eighth semester and the reason we drew the operating system boundary where we did in the first week.